第29章 詳解

29-E01 Free Variable 與最小化

推導

令$u=p-q$、$p,q\geq 0$，改最大化$-3p+3q-v$。等式變兩列$p-q-v\leq 2$、$-p+q+v\leq -2$，另$p-q+v\leq 6$。Slack為$s_1=2-p+q+v$、$s_2=-2+p-q-v$、$s_3=6-p+q-v$；所有$p,q,v,s\geq 0$。

原可行點映到$p=\max(u,0),q=\max(-u,0)$；任一新可行點映回$u=p-q$後恢復原等式與不等式，目標符號相反。直接代原等式$u=v+2$，另$2v+2\leq 6$給$0\leq v\leq 2$。原目標$3(v+2)+v=6+4v$，所以$v=0,u=2$、最小6；轉換後最大-6，可取$(p,q,v)=(2,0,0)$。

結論與檢核

最優原值6，並非把新最大值-6直接當原答案。此例最優u剛好非負，不構成通用加$u\geq 0$的理由；例如只給$u=-1$的可行題，額外非負會直接改成不可行。

29-E02 完整 Pivot 與對偶證書

推導

初始$x_3=10-2x_1-x_2$、$x_4=8-x_1-2x_2$、$z=4x_1+3x_2$。x_1的ratios是$10/2=5$與$8/1=8$，故x_3離。解出$x_1=5-x_2/2-x_3/2$，代回得$x_4=3-3x_2/2+x_3/2$，$z=20+x_2-2x_3$。

再選x_2：x_1列比值$5/(1/2)=10$、x_4列$3/(3/2)=2$，x_4離。新字典

$x_2=2+x_3/3-2x_4/3$； $x_1=4-2x_3/3+x_4/3$； $z=22-5x_3/3-2x_4/3$。

非基本$x_3=x_4=0$，得$x=(4,2)$，兩限制$8+2=10$與$4+4=8$，目標$16+6=22$。Dual $\min 10y_1+8y_2$，$2y_1+y_2\geq 4$、$y_1+2y_2\geq 3$、$y\geq 0$。由slack reduced costs讀$y=(5/3,2/3)$，兩式皆等號，目標$50/3+16/3=22$。

結論與檢核

兩次步長5、2，最優$(4,2)$、值22。每次都保留離基x_3或x_4的係數，最後dual同值提供獨立最佳性證書。

29-E03 零比值、平手與符號擾動

推導

第一題選x_1，x_3列ratio$0/1=0$、x_4列$2/1=2$，故x_3離，$\theta=0$。字典變$x_1=-x_3$、$x_4=2+x_3-x_2$、$z=-x_3+x_2$；點仍全decision0，$x_4=2$。選x_2，唯一正分母x_4列給$\theta=2$，得到$x_2=2+x_3-x_4$、$x_1=-x_3$、$z=2-x_4$。終止$x_3=x_4=0$，$x=(0,2)$，最優2。可行性也直接迫使$x_1=0$、$0\leq x_2\leq 2$。

第29章 詳解 （續）

第二題ratio為$2+\varepsilon$與$2+\varepsilon^2$；首個不同係數在ε項，前者1、後者0，所以後者較小，x_4離。並不是比較「ε^2的次方較大」；對正無窮小ε，ε^2比ε小。Pivot後$x_3=\varepsilon-\varepsilon^2+x_2+x_4$，在非基本0時常數0但符號值嚴格正。

結論與檢核

零步長仍是合法換basis；第一題最後點不同於初始，卻不是第一次退化pivot帶來的改善。Lex比較要從低次到高次，不可任意選浮點ε當成精確規則。

29-E04 從 Phase I 一路做到原最優

推導

原slacks $s_1=4-x$、$s_2=-2+x$，$x=0$時$s_2=-2$。Auxiliary $s_1=4-x+a$、$s_2=-2+x+a$、$w=-a$。選最負s_2離、a入，得$a=2-x+s_2$，$s_1=6-2x+s_2$，$w=-2+x-s_2$；此時$(x,a,s_1,s_2)=(0,2,6,0)$可行。

x入基的ratios是a列2、s_1列3，a離。得到$x=2+s_2-a$、$s_1=2-s_2+2a$、$w=-a$，基本點$x=2,a=0,s_1=2$。Auxiliary已最優0，刪a後$x=2+s_2$、$s_1=2-s_2$。恢復原$z=x=2+s_2$，讓s_2入、s_1離，$\theta=2$，得$s_2=2-s_1$、$x=4-s_1$、$z=4-s_1$，最優$x=4$。

改$x\le 1$時，初始化auxiliary $s_1=1-x+a$、$s_2=-2+x+a$，a入後$s_1=3-2x+s_2$、$a=2-x+s_2$、$w=-2+x-s_2$。x入的ratios3/2與2，s_1離。得$x=3/2-s_1/2+s_2/2$、$a=1/2+s_1/2+s_2/2$、$w=-1/2-s_1/2-s_2/2$，最優$-1/2$，原不可行。

結論與檢核

原題需分開Phase I最優0與原題最優4。修改題有獨立證書：$x\le 1$與$-x\le -2$相加得$0\le -1$。不能在初始$a=2$時就宣告不可行。

29-E05 無界 Ray 與無界區域

推導

初始$s=1-x_1+x_2$，$z=2x_1+x_2$。x_1入、s離，$\theta=1$，得到$x_1=1+x_2-s$、$z=2+3x_2-2s$。選x_2時基本列係數$a_{(1,2)}=-1$，不限制它增加；令$s=0$、$x_2=\theta$，便有$x=(1+\theta,\theta)$。

取$x^0=(1,0)$、$r=(1,1)$，$r\ge 0$，$A=(1,-1)$給$Ar=0\le 0$，$c^{\mathsf T}r=3>0$，所以每個$\theta\ge 0$都可行且目標$2+3\theta$無上界。改目標$-x_1-x_2$時，由$x\ge 0$即目標≤ 0，原點可行且達0。

結論與檢核

同一可行區域是否無界是集合性質；目標是否無界取決於c在可行ray上的增量，不能只看圖形延伸到無限遠。

第29章 詳解 （續）

29-E06 Potential 的模型前提

推導

全圖限制d_s=0、d_a≤−2、d_t−d_a≤3、d_t≤4，以及d_v−d_u≤−1、d_u−d_v≤−1。後兩式相加0≤−2，所以全圖LP不可行。這是不可達負cycle造成，與s到t無關。

限制到s可達子圖{s,a,t}後，沿s-a-t得到d_t≤1，取d_a=−2,d_t=1可行，最大值1。它等於兩條s-t路長min(−2+3,4)=1。錯加d_a≥0與d_a≤−2矛盾，直接排除所有正確potential。

結論與檢核

模型的每一條限制都有作用域；不能用不可達子圖的負cycle判定s到t沒有有限距離。修正後distance labels允許負值。

29-E07 共享容量與整數限制

推導

設上下路為s-a-t、s-b-t。對k=1,2，各arc量g^k≥0，在a,b分別守恆；source總流出與sink總流入各1。每arc共享限制$g^1_e+g^2_e\le 1$。

分數方案：每種貨在每條路各送1/2，每arc總1。整數方案：貨1上路送1、貨2下路送1，每arc同樣總1。兩方案都滿足模型，但第一方案不是「每貨只走單一路」；LP容許split，不代表它強制split。

唯一共享瓶頸容量1、需求各3/4時，每種貨單獨≤1可送；合併守恆迫使瓶頸總量3/2，違反容量。把兩份單貨解直接相加失敗，因它們在同一個限制分母競爭。

結論與檢核

此例恰有整數解不能推出任意multicommodity LP有整數解。整數量與不可分路徑都是額外限制；它們也不同，例如需求2的整數貨量可分成兩條路各1，仍不是單一路。

29-E08 Complementary Slackness 與條件邊界

推導

Dual為min$4y_1+2y_2$，$y_1+y_2\ge 2$、$y_1\ge 1$、$y\ge 0$。x=(2,2)滿足$x_1+x_2=4$、$x_1=2$，值6；y=(1,1)滿足兩dual等式，值4+2=6。

Gap分解：primal slacks為(0,0)，dual slacks為(0,0)，故$y^{\mathsf{T}}(b-Ax)+x^{\mathsf{T}}(A^{\mathsf{T}}y-c)=0+0=0$，兩側最優。對x'=(1,3)，第一slack0、第二slack1，目標5；與同一y的gap=1，正好是$y_2(2-x_1)=1\cdot 1$，不符合互補。

第29章 詳解 （續）

嚴格限制例的每個可行x都有更大可行值(x+1)/2<1，故沒有最大點；目標又有上界1，所以既非infeasible、也非向上unbounded、也非達finite optimum。這不反駁定理，因定理要求非嚴格LP限制。

結論與檢核

證書必同時檢查可行、目標與互補乘積。上確界與被某點達到的最大值是不同概念。

原創驗算程式附錄: phase_one_certificate.py

```
"""Verify the Phase I states and a matching primal-dual certificate."""


def objective(x1, x2):
    return 2 * x1 + x2


def run():
    phase_rows = [
        (0.0, 0.0, 0.0, 5.0),
        (5.0, 0.0, 0.0, 0.0),
        (0.0, 5.0, 0.0, 0.0),
    ]
    assert all(
        abs(x1 + x2 - x3 + artificial - 5) < 1e-12
        and min(x1, x2, x3, artificial) >= 0
        for x1, x2, x3, artificial in phase_rows
    )
    assert objective(*phase_rows[1][:2]) == 10
    assert objective(*phase_rows[2][:2]) == 5

    primal = objective(0, 5)
    dual_y = 1
    dual_bound = 5 * dual_y
    assert primal == dual_bound == 5
    assert 1 * dual_y <= 2
    assert 1 * dual_y <= 1
    print("PASS phase-I artificial=5,w=0 primal=5 dual=5 ratio_rows_include_zero=True")
    return primal, dual_bound


if __name__ == "__main__":
    run()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第29章 詳解 （續）

原創驗算程式附錄: simplex_exact.py

```
"""Exact two-phase dictionary simplex with symbolic lex RHS perturbation.

Fractions and polynomial coefficient tuples are a reference for mathematical
steps, not a production floating-point optimizer or a timing benchmark.
"""
from fractions import Fraction as F

class Dictionary:
    def __init__(self,B,N,b,a,c,v=0):
        self.B=list(B);self.N=list(N)
        self.b={i:F(b[i]) for i in B};self.a={(i,j):F(a.get((i,j),0)) for i in B for j in N}
        self.c={j:F(c.get(j,0)) for j in N};self.v=F(v);self.trace=[]
    def point(self):return {**{j:F(0) for j in self.N},**self.b}
    def pivot(self,l,e):
        p=self.a[l,e];assert p
        oldB=list(self.B);oldN=list(self.N);newN=[j for j in oldN if j!=e]+[l]
        newb={e:self.b[l]/p};newa={(e,l):1/p}
        for j in oldN:
            if j!=e:newa[e,j]=self.a[l,j]/p
        for i in oldB:
            if i==l:continue
            newb[i]=self.b[i]-self.a[i,e]*newb[e]
            newa[i,l]=-self.a[i,e]/p
            for j in oldN:
                if j!=e:newa[i,j]=self.a[i,j]-self.a[i,e]*newa[e,j]
        newv=self.v+self.c[e]*newb[e]
        newc={l:-self.c[e]/p}
        for j in oldN:
            if j!=e:newc[j]=self.c[j]-self.c[e]*newa[e,j]
        if hasattr(self,'lex'):
            old=self.lex;row=tuple(x/p for x in old[l]);self.lex={e:row}
            for i in oldB:
                if i!=l:self.lex[i]=tuple(x-self.a[i,e]*y for x,y in zip(old[i],row))
        self.B=[e if i==l else i for i in oldB];self.N=newN
        self.b=newb;self.a=newa;self.c=newc;self.v=newv

self.trace.append({'enter':e,'leave':l,'b':dict(self.b),'a':dict(self.a),'c':dict(self.c),'v'
:self.v})
    def optimize(self):
        assert all(b>=0 for b in self.b.values())
        m=len(self.B)
        self.lex={i:(self.b[i],)+tuple(F(int(k==r)) for k in range(m)) for r,i in
enumerate(self.B)}
        visited=set()
        while True:
            state=frozenset(self.B);assert state not in visited;visited.add(state)
            choices=[j for j in self.N if self.c[j]>0]
            if not choices:del self.lex;return 'optimal',None
            e=min(choices);rows=[i for i in self.B if self.a[i,e]>0]
            if not rows:
                ray={j:F(0) for j in self.N};ray[e]=F(1)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第29章 詳解 （續）

```
                ray.update({i:-self.a[i,e] for i in self.B});del self.lex;return
'unbounded',ray
            l=min(rows,key=lambda i:tuple(x/self.a[i,e] for x in self.lex[i]))
            self.pivot(l,e)
            assert all(b>=0 for b in self.b.values())
            assert all(next((x for x in row if x),0)>0 for row in self.lex.values())

def solve(A,b,c):
    A=[[F(x) for x in row] for row in A];b=list(map(F,b));c=list(map(F,c))
    m=len(b);n=len(c);B=list(range(n,n+m));N=list(range(n))
    d=Dictionary(B,N,dict(zip(B,b)),{(n+i,j):A[i][j] for i in range(m) for j in
N},dict(enumerate(c)))
    phase1=[]
    if m and min(b)<0:
        artificial=-1;d.N.append(artificial);d.c={j:F(0) for j in d.N};d.c[artificial]=F(-1)
        for i in d.B:d.a[i,artificial]=F(-1)
        l=min(d.B,key=lambda i:d.b[i]);d.pivot(l,artificial)
        status,_=d.optimize();assert status=='optimal';phase1=list(d.trace)
        if d.v<0:return {'status':'infeasible','phase1':phase1,'auxiliary_value':d.v}
        assert d.v==0
        if artificial in d.B:
            rows=[j for j in d.N if d.a[artificial,j]]
            if rows:d.pivot(artificial,min(rows))
            else:
                d.B.remove(artificial);d.b.pop(artificial)
                d.a={ij:x for ij,x in d.a.items() if ij[0]!=artificial}
        if artificial in d.N:d.N.remove(artificial)
        d.a={ij:x for ij,x in d.a.items() if artificial not in ij}
        d.v=sum(c[i]*d.b[i] for i in d.B if 0<=i<n)
        d.c={j:(c[j] if 0<=j<n else F(0))-sum(c[i]*d.a[i,j] for i in d.B if 0<=i<n) for j in
d.N}
        d.trace=[]
    status,ray=d.optimize();q=d.point();x=[q.get(j,F(0)) for j in range(n)]
    assert all(sum(a*t for a,t in zip(row,x))<=rhs for row,rhs in zip(A,b))
    if status=='unbounded':
        r=[ray.get(j,F(0)) for j in range(n)]
        assert all(v>=0 for v in r) and all(sum(a*t for a,t in zip(row,r))<=0 for row in A)
        assert sum(a*t for a,t in zip(c,r))>0
        return {'status':status,'x':x,'ray':r,'phase1':phase1,'phase2':d.trace}
    value=sum(a*t for a,t in zip(c,x));assert value==d.v
    y=[-d.c.get(n+i,F(0)) for i in range(m)]
    assert all(v>=0 for v in y)
    assert all(sum(A[i][j]*y[i] for i in range(m))>=c[j] for j in range(n))
    assert sum(b[i]*y[i] for i in range(m))==value
    return {'status':status,'x':x,'value':value,'dual':y,'phase1':phase1,'phase2':d.trace}

if __name__=='__main__':
    for args in [([[1,2],[3,1]],[8,9],[3,2]),([[-1,-1]],[-5],[-2,-1]),([[1],[-1]],[1,-2],
[1]),([[-1]],[-1],[1])]:
        print(solve(*args))
```

第29章 詳解 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study